Software User Manual

ICU Sepsis Dynamic Subtype Discovery and Analysis System

Wang Ruike

March 18, 2026

Contents

1 Software Overview **2**

2 System Requirements **2**

3 Installation Steps **2**
3.1 Obtain the Project Code . . . 2
3.2 Install Python Dependencies . . . 2
3.3 Verify Installation . . . 3

4 Project Directory Structure **3**

5 Input Data Format **3**
5.1 Simulated Data (Default) . . . 3
5.2 Real Data Interface (MIMIC-IV / eICU) . . . 4

6 Configuration File **4**
6.1 Data Source . . . 4
6.2 Preprocessing . . . 4
6.3 Clustering . . . 4
6.4 Dimensionality Reduction . . . 5

7 How to Run **5**
7.1 Basic Execution . . . 5
7.2 Command-Line Arguments . . . 5
7.3 Usage Examples . . . 5

8 Output Description **6**
8.1 Visualization Charts (`outputs/figures/`) . . . 6
8.2 Evaluation Reports (`outputs/reports/`) . . . 6
8.3 Sample Output Figures . . . 6

9 Frequently Asked Questions **7**

10 Important Notes **8**

1 Software Overview

This software is a dynamic subtype discovery and analysis system designed for ICU sepsis patient time-series data. Based on electronic health record (EHR) time-series data, the system performs standardized preprocessing, statistical feature extraction, dimensionality reduction, and cluster analysis to automatically discover dynamic subtypes among sepsis patients.

Key features include:

- Loading and preprocessing of simulated or real ICU sepsis patient time-series data
- Feature extraction across multiple time windows with multiple statistical measures
- Automatic optimal cluster number search and multi-method clustering comparison
- Clustering quality evaluation, survival stratification analysis, and subtype clinical profiling
- Automatic generation of 7 types of visualization charts
- Structured evaluation report output (JSON + plain text)

2 System Requirements

Table 1: Minimum and recommended system requirements.

Item	**Requirement**
Operating System	macOS / Linux / Windows
Python Version	3.9 or above
Memory	4 GB minimum (8 GB recommended)
Disk Space	500 MB minimum (including dependencies)
GPU	Not required (MVP version)

3 Installation Steps

3.1 Obtain the Project Code

Copy the project directory to your local working directory:

```
cd /path/to/project
```

3.2 Install Python Dependencies

```
pip install -r requirements.txt
```

Key dependencies:

Table 2: Key Python dependencies.

Package	Version	Purpose
numpy	≥1.24.0	Numerical computation
pandas	≥2.0.0	Data processing
scipy	≥1.10.0	Scientific computation
pyyaml	≥6.0	Configuration file parsing
scikit-learn	≥1.3.0	Clustering and evaluation
umap-learn	≥0.5.3	Dimensionality reduction
matplotlib	≥3.7.0	Visualization
lifelines	≥0.27.0	Survival analysis (recommended)
duckdb	≥0.9.0	MIMIC-IV data pipeline

3.3 Verify Installation

```
cd src
python main.py --n-patients 100 --skip-vis
```

If the output displays `"All steps completed!"`, the installation is successful.

4 Project Directory Structure

```
project/
+-- config/
| +-- config.yaml # Global configuration
+-- data/
| +-- raw/ # Raw data
| +-- processed/ # Preprocessed cached data
| +-- demo/ # Demo data
+-- docs/ # Project documentation
+-- outputs/
| +-- figures/ # Visualization output (PNG)
| +-- models/ # Saved model files
| +-- reports/ # Evaluation reports (JSON + TXT)
+-- src/
| +-- main.py # Main entry point
| +-- data_loader.py # Data loading
| +-- preprocess.py # Preprocessing module
| +-- feature_engineering.py # Feature extraction
| +-- representation_model.py# Representation learning
| +-- clustering.py # Clustering module
| +-- evaluation.py # Evaluation module
| +-- visualization.py # Visualization module
| +-- utils.py # Shared utilities
+-- requirements.txt
+-- README.md
```

5 Input Data Format

5.1 Simulated Data (Default)

The system includes a built-in simulated data generator, allowing it to run without external data. The simulator produces time-series data for patients with 4 sepsis subtypes based on clinically reasonable parameters.

5.2 Real Data Interface (MIMIC-IV / eICU)

When using real data, the following format is required:

Time-series data (3D tensor):

- Shape: `(n_patients, n_timesteps, n_features)`
- Data type: `float64`
- Missing values: represented by `NaN`

Patient information table (DataFrame):

Table 3: Patient information table schema.

Column	Type	Description
patient_id	str	Unique patient identifier
mortality_28d	int	28-day mortality (0/1)
icu_los	float	ICU length of stay (hours)
shock_onset	int	Shock occurrence (0/1)
age	int	Age
gender	str	Gender (M/F)

6 Configuration File

The configuration file is located at `config/config.yaml` in YAML format.

6.1 Data Source

```
data:
  source: "simulated" # "simulated" | "mimic" | "eicu"
  simulated:
    n_patients: 500
    n_timesteps: 48
    n_subtypes: 4
    missing_rate: 0.15
    random_seed: 42
```

6.2 Preprocessing

```
preprocess:
  missing_strategy: "forward_fill_then_median"
  outlier_method: "clip"
  outlier_sigma: 4.0
  normalization: "standard"
```

6.3 Clustering

```
clustering:
  method: "kmeans"
  k_range: [2, 8]
  optimal_k_criterion: "silhouette"
```

6.4 Dimensionality Reduction

```
reduction:
  method: "umap" # "umap" | "tsne" | "pca"
  n_components: 2
  umap:
    n_neighbors: 15
    min_dist: 0.1
```

7 How to Run

7.1 Basic Execution

```
cd project/src
python main.py
```

This runs the full analysis pipeline with default settings (500 simulated patients).

7.2 Command-Line Arguments

Table 4: Available command-line arguments.

Argument	Default	Description
`--config`	config.yaml	Custom config file path
`--n-patients`	500	Number of simulated patients
`--source`	simulated	Data source
`--method`	kmeans	Clustering method
`--k`	auto	Cluster count
`--reduction`	umap	Reduction method
`--seed`	42	Random seed
`--skip-vis`	false	Skip visualization
`--compare-methods`	false	Multi-method comparison

7.3 Usage Examples

```
# 1000 patients, GMM clustering
python main.py --n-patients 1000 --method gmm

# K=4, t-SNE reduction
python main.py --k 4 --reduction tsne

# Quick run (skip visualization)
python main.py --n-patients 200 --skip-vis

# Multi-method clustering comparison
python main.py --compare-methods

# MIMIC-IV data source
python main.py --source mimic
```

8 Output Description

8.1 Visualization Charts (`outputs/figures/`)

Table 5: Generated visualization files.

File	Description
`missing_pattern.png`	Missing rate distribution per variable
`cluster_scatter.png`	Cluster scatter plot after UMAP/t-SNE reduction
`k_selection.png`	Optimal K selection curves (Silhouette, CH, DB)
`subtype_heatmap.png`	Subtype feature profile heatmap
`survival_curves.png`	Kaplan-Meier survival curves per subtype
`trajectory_comparison.png`	Key variable trajectory comparison
`summary_dashboard.png`	Comprehensive results dashboard (2×2)

8.2 Evaluation Reports (`outputs/reports/`)

Table 6: Generated report files.

File	Format	Content
`evaluation_report.json`	JSON	Complete evaluation data (machine-parseable)
`evaluation_summary.txt`	Text	Human-readable summary

8.3 Sample Output Figures

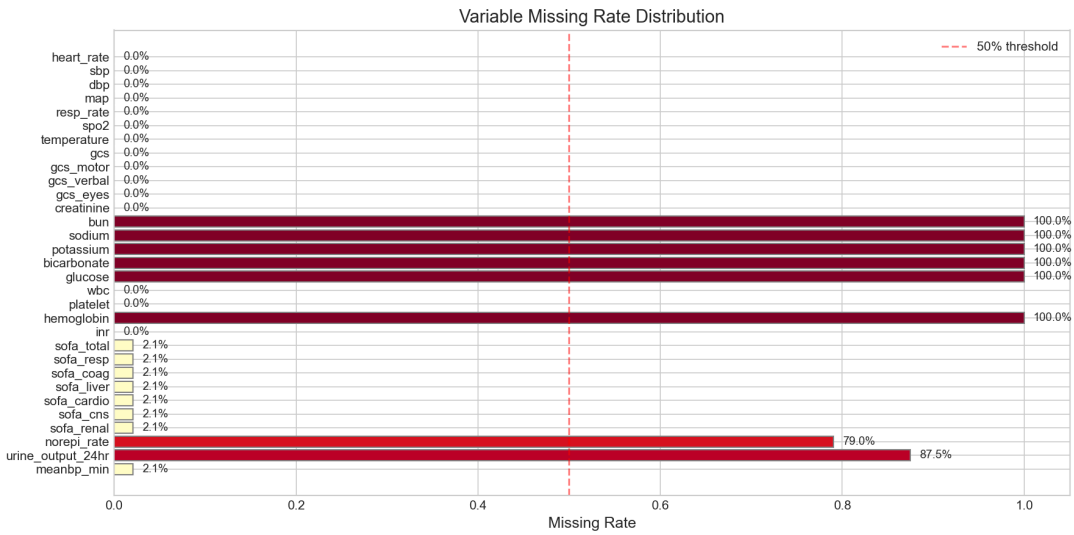

Figure 1: Example output: Missing rate distribution across clinical variables.

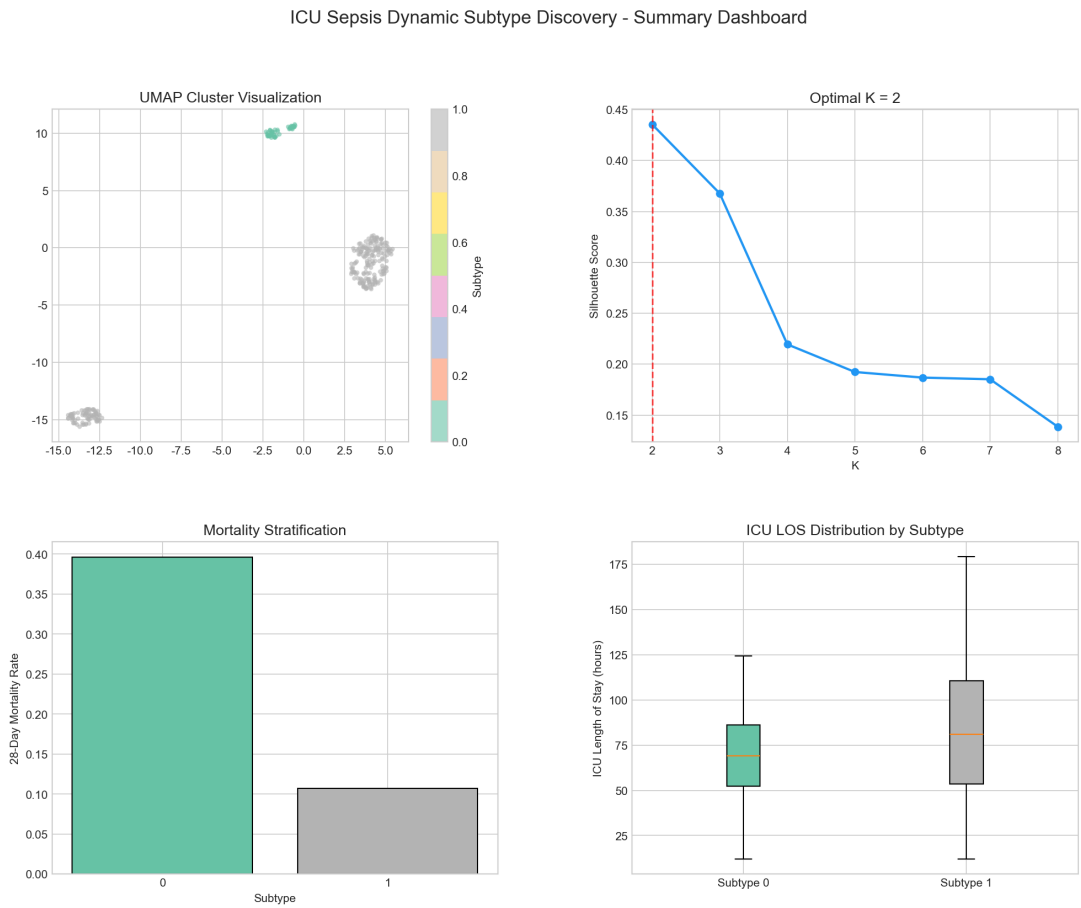

Figure 2: Example output: Summary dashboard with cluster scatter, K selection, mortality, and LOS distribution.

9 Frequently Asked Questions

Q1: `ModuleNotFoundError` at runtime

A: Ensure all dependencies are installed: `pip install -r requirements.txt`

Q2: UMAP dimensionality reduction is slow

A: UMAP may be slow with >5000 patients. Use `--reduction pca` as a faster alternative, or reduce patient count for quick validation.

Q3: How to use real MIMIC-IV data?

A: Complete PhysioNet credentialing at `https://physionet.org/content/mimiciv/`, download the data, then run:

```
python import_to_duckdb.py --data-dir ../mimic-iv-data
python run_concepts.py
python build_analysis_table.py
python main.py --source mimic
```

Q4: Clustering results differ between runs

A: Ensure a random seed is set (default 42). All random processes are managed uniformly via the `set_global_seed()` function.

Q5: How to change the number of clusters?

A: Use `--k 4` to force $K = 4$, or modify `k_range` in `config.yaml` for the auto-search range.

10 Important Notes

1. For the first run, use default settings with a smaller patient count (200–300) for validation.
2. Configuration changes take effect immediately on the next run; no reinstallation needed.
3. Results in `outputs/` are overwritten on subsequent runs; back up manually if needed.
4. Simulated data is for development and validation only; use real data for formal reports.
5. The survival analysis feature requires `lifelines`; without it, the system falls back to chi-square tests.